

Fast Errors or Slow Effort? Dual-Process Signatures in Human and LLM Code Understanding

Abstract—Code obfuscation is designed to impede comprehension, but it is unclear whether different obfuscation techniques disrupt understanding through a single mechanism or through distinct cognitive routes. We test a dual-process account in which adversarial identifier renaming and control-flow obfuscation disrupt comprehension through distinct effortful routes: renaming as Stroop-like interference that readers detect and suppress at the cost of a strategy switch, and control-flow as System 2 overload that degrades comprehension through sustained tracing. In a study of 73 participants spanning a range of JavaScript and Python experience, we measure accuracy, completion time, confidence, calibration, and answering strategy across obfuscation tiers (L0, L1b, L2) under manipulations of time pressure and concurrent cognitive load. We first replicate prior obfuscation effects on human accuracy and time, then examine whether renaming and control-flow obfuscation yield the predicted divergent signatures, whether time pressure and dual-task load shift readers between System 1 and System 2 strategies, and whether performance reveals a cognitive limit analogous to chain-of-thought length effects. Finally, we ask whether large language models reproduce the same accuracy, confidence, and failure patterns, testing the alignment between human and model comprehension of obfuscated code.

I. INTRODUCTION

Code obfuscation preserves a program’s semantics while stripping the lexical and structural cues a reader relies on, making it a clean probe of comprehension: does an agent—human or model—understand program logic, or lean on surface patterns like meaningful identifiers and recognizable control flow? The interesting question is not whether obfuscation makes code harder but *how* removing cues breaks understanding, and that “how” is not one mechanism. Difficulty is usually assumed to grow monotonically with obfuscation strength, yet this already fails for humans, where some renaming transformations match or exceed unobfuscated accuracy [1]. Different transformations attack different things: adversarial identifier renaming injects plausible-but-wrong semantics that interfere with the reader’s heuristic read—Stroop-like, and detected as such—while control-flow alteration dismantles the structural scaffolding behind a reader’s mental model, forcing sustained, effortful tracing—so there is no reason to expect these *kinds* of difficulty to fail through a common route.

For example, the Tribonacci function evaluated at `myFunct(14)` requires summing fourteen terms by hand to reach 927927 927, which exceeds working memory. A reader who *recognizes* the recurrence answers correctly; a reader who tries to *trace* it exhausts working memory and runs out of time without answering. Here fast pattern recognition is the *winning* strategy and deliberate tracing is the trap—the inversion of the usual “slow down and you’ll get it right” story. The same

transformation is trivial for one reader and a trap for another, depending on which cognitive route they take. Measuring that route, not just the outcome, is what this paper does. We use two lenses. Dual-process theory and Schulte’s Block Model [2].

This builds on two prior studies and closes gaps neither could address. Nguyen et al. [1] showed obfuscation impedes human comprehension and shifts readers toward System 2, but inferred that shift only *post hoc* from response time, with no confidence, calibration, or strategy measures—so it could not separate a confident trap from a slow overload. A companion model study [3] found the matching machine signatures, but neither manipulated the reader’s cognitive resources nor placed human and model failures on the same instrument. We re-run the human study on an independent sample with explicit confidence, calibration, and strategy measures; add time-pressure and dual-task levers that actively push readers between regimes; probe for a cognitive breakdown point; and compare humans and models on matched stimuli, using function-level output prediction in Python and JavaScript across three tiers (L0, L1b, L2).

This paper makes the following contributions:

- **A dual-route account.** We formalize and operationalize the hypothesis that renaming and control-flow obfuscation fail through distinct *effortful* routes: atom-level interference that forces a strategy switch versus System 2 relational/macro-level overload.
- **Refined human replication.** On 73 participants, the reliable per-tier penalty is carried by control-flow rather than renaming, concentrated in JavaScript, and heaviest on the least experienced readers.
- **Human-model alignment.** Reasoning-tuned LLMs reproduce the human accuracy shape, the control-flow effort signature, and a bounded reasoning-budget plateau—while coder- and instruction-tuned models do not—and the apparent renaming divergence dissolves once both populations are scored on a shared grader ruler.

II. BACKGROUND

A. Code Obfuscation

Prior work categorizes obfuscation techniques by transformation properties, most notably distinguishing layout-based from control-flow-based obfuscations [4]. Identifier renaming is a layout-based obfuscation, while control-flow flattening is a control-flow-based obfuscation [5]. We adopt these obfuscation levels to align with Nguyen *et al.* [1]’s human study on code understandability under obfuscation.

L0: Original code snippet, no obfuscation applied.

L1b: Adversarial Renaming. Where plain renaming (L1) removes semantic cues, L1b replaces identifiers with semantically meaningful but *misleading* names, attaching a plausible-but-wrong meaning to a familiar-looking name to induce incorrect mental models—the Stroop-like interference we formalize in Section II-B.

L2: Control-Flow Alteration. Decouples control logic from its original syntactic structure, decomposing code into smaller blocks and introducing artificial control constructs (e.g. dispatchers or indirect jumps) that make the original execution order difficult to reconstruct [6].

B. Dual-Process Theory and the Block Model

Dual-process theory distinguishes System 1 (fast, intuitive, heuristic) from System 2 (slow, analytic, deliberate) [7]. The Default-Interventionist model holds that readers default to System 1 and engage System 2 only when that default fails. We treat obfuscation as a barrier to the default mode that forces this more expensive intervention.

Where dual-process theory describes *how* a reader processes, Schulte’s Block Model [2] describes *what* is processed, as a hierarchy from atoms (keywords, variables) through blocks (e.g. a loop body) and relations (data flow between blocks) to macro structure (the overall algorithm). Expert comprehension is top-down, anchored on familiar “beacons” (recognizable names, standard structures); novice comprehension is bottom-up, statement-by-statement. Obfuscation removes beacons and flattens this hierarchy, forcing readers back to slow, error-prone atom-level analysis.

The two lenses yield the paper’s hypothesis. Adversarial renaming operates at the *atom* level, corrupting the identifier-to-meaning mapping and producing Stroop-like interference: the misleading name is recognized and must be suppressed, forcing an effortful switch away from the heuristic read. Control-flow alteration operates at the *relational* and *macro* levels, forcing reconstruction of execution state through dispatcher-mediated indirection and imposing System 2 overload that degrades accuracy as effort rises. This atom-versus-relation mapping makes the two routes separable and grounds RQ2 and RQ4.

Two controlled studies frame ours: Nguyen et al. [1] established that obfuscation impedes human comprehension and shifts strategy, and a companion study [3] found matching model signatures. Neither separated the two routes with direct confidence and strategy evidence, manipulated the reader’s cognitive resources, or placed human and model failures on a single instrument — the gaps we close.

III. METHODOLOGY

The study pairs a controlled human experiment with a parallel model evaluation, joined by a common output-prediction task and obfuscation tiers. The human experiment crosses tier with two cognitive-resource manipulations—time pressure and dual-task load—adding confidence, calibration, and strategy measures to the inherited accuracy and response-time ones.

The model evaluation runs the same task across LLM families under prompting and token-budget manipulations that mirror the human levers. Human survey structure is shown in Fig. 1.

A. Stimuli and Obfuscation Tiers

Stimuli are function-level output-prediction items in Python and JavaScript, following Nguyen et al. [1]: HumanEval-X programs with cyclomatic complexity four to eight and fewer than fifteen lines—non-trivial yet hand-traceable. Each base snippet is paired with a fixed input held constant across tiers. Obfuscated variants are generated with ObfuXtreme (Python) and javascript-obfuscator (JavaScript), each configured to isolate one transformation and validated by execution for functional equivalence. Adversarial identifiers for L1b are drawn from a held-out CodeSearchNet split and matched by semantic domain so the misleading names stay plausible. Each item appears at three tiers—L0, L1b, and L2—and no participant sees a base snippet at more than one tier, preventing carryover. The study draws on six base snippets—three JavaScript (items 63, 58, 163) and three Python (36, 74, 104). Each carries a fixed route label (Section V): two are S1-amenable (63/JS, 36/Py) and four are S2-amenable (58/JS, 163/JS, 104/Py, 74/Py). Each base snippet appears at all three tiers (L0, L1b, L2); every participant sees all six snippets, each at one randomly assigned tier, and no participant sees a base snippet at more than one tier, preventing carryover.

B. Dual-Task Conditions

We use three conditions: closely-grouped, loosely-grouped, and control. Following the dual-task literature, participants memorize a sequence of images before answering the comprehension question, then recall the sequence afterward. In the closely-grouped condition the images form a coherent set (e.g., a series of animals), whereas in the loosely-grouped condition they are unrelated. The control condition has no secondary memorization task.

C. Participants

73 participants separately reported prior Python and JavaScript experience as none, under one year, or one year or more (joint distribution in Fig. 2); for analyses needing an ordered skill variable, these self-reports map to beginner, intermediate, and advanced tiers.

JavaScript Experience	Python Experience			Total
	None	< 1 year	≥ 1 year	
None	7	8	5	20
< 1 year	4	10	12	26
≥ 1 year	2	7	18	27
Total	13	25	35	73

Fig. 2: Participant Experience Levels

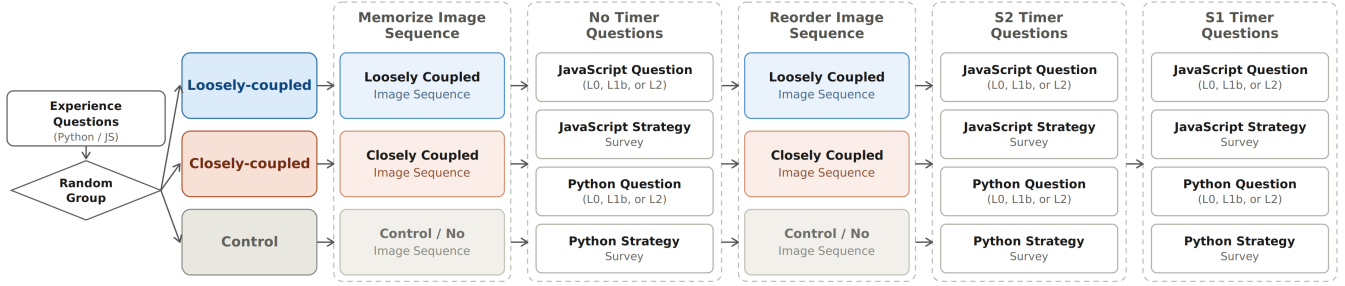


Fig. 1: Survey Workflow

D. Procedure

Participants are randomly assigned to control, loosely-coupled, or closely-coupled groups. The two non-control groups memorize an image sequence—loosely or closely related by assignment—while the control group sees none. This is the *dual-task load lever*: it occupies working memory during comprehension, reducing the System 2 resources available for tracing, with the closely- versus loosely-coupled contrast varying interference.

All participants then complete an untimed round of two output-prediction questions (one JavaScript, one Python, each from tier L0, L1b, or L2), each followed by a strategy survey. They next reorder the memorized images, then complete two further timed rounds of the same paired structure. The untimed-versus-timed contrast is the *time-pressure lever*: time pressure pushes readers toward fast System 1 processing, most dangerous where a fast, pattern-driven read is likely wrong. Both levers are manipulated directly rather than inferred from response time, letting us attribute strategy shifts to the manipulation rather than to unobserved item difficulty.

E. Measures

Inherited measures. Response time is recorded per question, and programming experience is the self-reported Python and JavaScript categories above. Correctness is established by human grading rather than automatic output matching: two annotators independently labeled each response as correct, partial, or incorrect against the reference output. The reported label is the consensus of the two graders; disagreements are resolved by an automated (LLM) tiebreak, with a single-grader fallback where only one annotation is available.

New measures. Participants report *confidence* on a 1–10 scale, enabling a *calibration* analysis of accuracy against confidence, summarized by the reliability curve and expected calibration error. They also report *strategy*, coded into five categories along the dual-process spectrum—recognized the pattern and guessed (fast / System 1); confused by names, then traced (mixed); ignored names and traced every line (System 2); ran out of time; and other. From confidence and correctness we derive a *high-confidence incorrect* (HCI) flag, and for the cross-population comparison we code each response’s *reasoning mode* as heuristic, mixed, tracing, or none.

F. Model Evaluation Setup

The model evaluation reuses the task, tiers, and matched stimuli for direct human–model comparison [3]. We evaluate reasoning-distilled, instruction-tuned, and coder-tuned families across a range of parameter counts, separating the effect of scale from that of tuning type. Prompting manipulates processing style: an instructed-System 1 condition elicits rapid, non-deliberative answers and an instructed-System 2 condition explicit line-by-line tracing (for full prompts, see [3])—the model analog of the human time-pressure lever. To probe the cognitive limit, we sweep the reasoning token budget from 256 to 10,000 tokens, isolating where added reasoning length stops buying accuracy. Because models report no confidence, we read out four probability-based proxies—exponentiated mean log-probability, self-certainty, inverse perplexity, and one minus normalized entropy—and compare their calibration against the human curve. Human–model alignment is computed on the matched items answered by both populations.

IV. RESEARCH QUESTIONS

RQ1 (Replication). Does increasing obfuscation reduce comprehension accuracy and increase completion time, and how does prior programming experience (JavaScript, Python) moderate these effects?

RQ2 (Confusion routes). Do adversarial renaming (L1b) and control-flow obfuscation (L2) disrupt comprehension through distinct effortful routes—interference-driven strategy switching versus System 2 overload—or does renaming instead spring the covert high-confidence trap that Paper 2 found in models? And how do these human routes compare with Paper 2’s HCI and dispatcher-complexity signatures?

RQ3 (Levers and limits). How do time pressure (a System 1 lever) and dual-task load (a System 2 lever) shift reasoning strategy and affect accuracy, confidence, and calibration across obfuscation tiers, and is there a breakdown point analogous to Paper 2’s chain-of-thought-length / Goldilocks effect?

RQ4 (Model alignment). Do large language models reproduce the human accuracy, confidence, and breakdown patterns across obfuscation tiers, including the renaming and control-flow signatures?

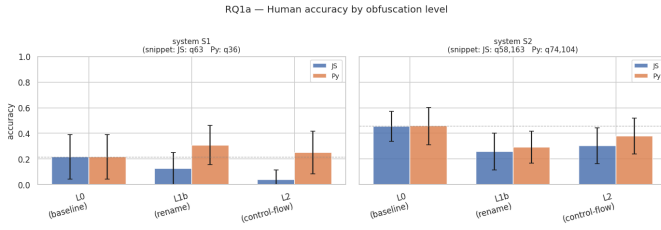


Fig. 3: Bar graph of accuracy by obfuscation level (L0, L1b, L2), split by item route (S1, S2) and language (JavaScript, Python). Error bars are 95% CIs.

TABLE I: GEE odds ratios for a correct answer (log scale; 1 = no effect). * indicates a 95% CI clear of 1.

Contrast	OR	95% CI
L1b vs L0	0.74	[0.48, 1.14]
L2 vs L0	0.57*	[0.46, 0.96]
S2 vs S1	2.45*	[1.64, 3.65]
Py vs JS	1.35	[0.96, 1.90]
control vs closely-coupled	0.63	[0.26, 1.53]
loosely- vs closely-coupled	0.84	[0.36, 1.99]

V. RQ1: REPLICATION OF OBFUSCATION EFFECTS IN HUMANS

Paper 1 reported that obfuscation lowers human comprehension accuracy and raises response time, but non-uniformly: JavaScript degraded as expected while Python was irregular, and the penalty fell hardest on less-experienced readers. Using a model that respects the repeated-measures design, plus strategy and confidence measures absent from the original, we reproduce the *direction* of that effect but not its uniformity. The reliable per-tier penalty is carried by control-flow (L2), not renaming (L1b); it is concentrated in JavaScript and heaviest on the least-experienced readers. Moving past Paper 1, Python’s apparent “irregularity” is not a language quirk but an item effect: once items are split by the cognitive route they reward, renaming helps where a fast pattern-read suffices and hurts where tracing is required, so it produces no reliable *accuracy* effect overall—a null that motivates RQ2.

Each item carries a fixed S1-/S2-amenable label (two S1: 63/JS, 36/Py; four S2: 58/JS, 163/JS, 104/Py, 74/Py) reflecting whether its output is more readily recovered by pattern recognition or by line-by-line tracing.

Pooled across language, route, and load, accuracy falls from a baseline of 0.39 to 0.26 under renaming and 0.28 under control-flow—both obfuscated tiers sit below baseline—so at the coarsest level obfuscation impedes comprehension. But the pattern is not the monotonic ramp the literature assumes (Fig. 3): only S1 JavaScript declines monotonically, S1 Python inverts it, and both languages dip-then-recover under S2. Because these means pool unevenly across participants and items, we fit a model that accounts for repeated measures.

The GEE adjusts each effect for repeated answers per participant; each is an odds ratio (OR), with an interval excluding 1 marking a reliable effect. Only two hold up (Table I): control-flow reliably lowers accuracy (L2, OR = 0.57) and the S1/S2

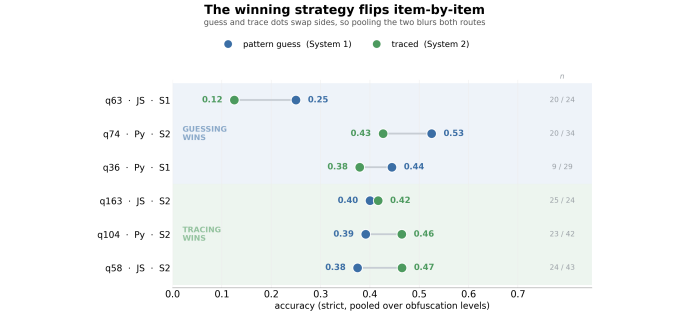


Fig. 4: Dot plot of the winning strategy (pattern-recognition vs. tracing) per question.

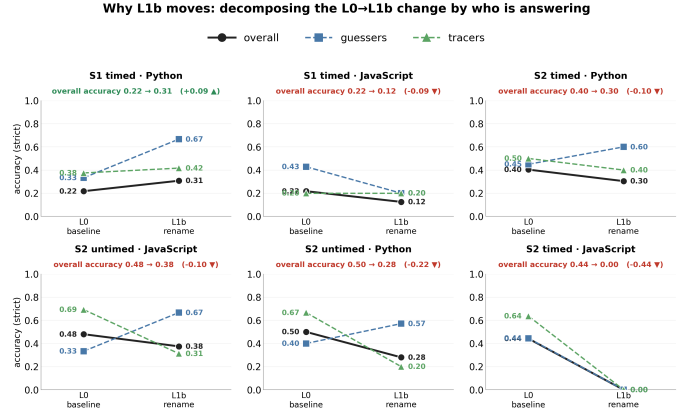


Fig. 5: Line plot of the L0→L1b accuracy change by answering strategy (guess, trace, overall) within each (route × timing × language) cell.

item split reliably raises it (OR = 2.45, odds more than doubled). The other four lean in plausible directions but their intervals cross 1—renaming (L1b, OR = 0.74) included. The headline is a selective replication: obfuscation lowers accuracy, but the reliable per-tier effect comes from control-flow, not renaming, which raises why renaming looks irregular in the raw means.

The winning strategy flips with the item—a fast pattern-read wins on S1-amenable items, tracing wins on S2-amenable items, and using the wrong strategy for an item costs accuracy (Fig. 4)—so pooling the two routes blurs both.

This route-dependence is what drives L1b’s apparent swings. Decomposing the L0→L1b accuracy change by who is answering (Fig. 5) shows the net effect of renaming flips sign with item route: small or positive on S1-amenable items (e.g. +0.09 on timed S1 Python) and sharply negative on S2-amenable items (e.g. −0.44 on timed S2 JavaScript). Renaming does not uniformly help Python or hurt JavaScript; it spares items a reader can recognize and punishes items a reader must trace. Averaging the two routes is what cancels into the unreliable pooled L1b effect from the GEE.

Aggregated to the language level (Fig. 6), the pooled effect is carried almost entirely by JavaScript: JS accuracy falls from ≈ 0.40 at L0 to ≈ 0.20 at both obfuscated tiers ($r = -0.19$,

Accuracy by obfuscation level, by programming language

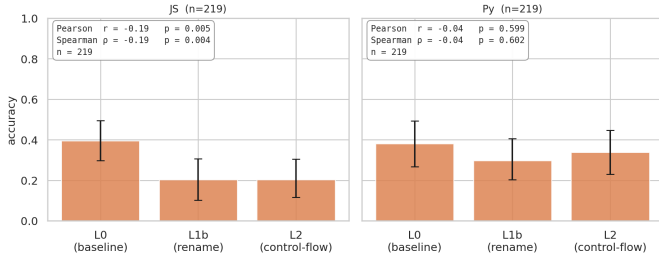


Fig. 6: Bar graph of accuracy by obfuscation level, split by language (JavaScript, Python). Error bars are 95% CIs.

Accuracy by obfuscation level across prior-experience tiers

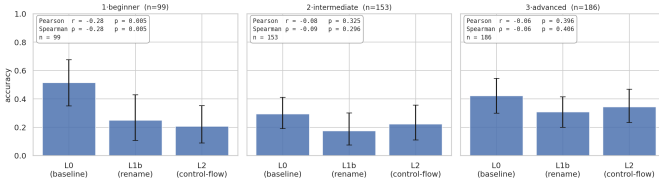


Fig. 7: Bar graph of accuracy by obfuscation level within prior-experience tiers (beginner, intermediate, advanced). Error bars are 95% CIs.

$p = .005$; $n = 219$), while Python is statistically flat ($r = -0.04$, $p = .60$) and partially recovers at L2. This directly replicates Paper 1 and fits the item-route account: the language difference tracks how many of each language’s items reward tracing.

The penalty is also experience-graded (Fig. 7). For beginners, accuracy falls steeply from ≈ 0.51 at L0 to ≈ 0.21 at L2—the only tier with a reliable negative association ($r = -0.28$, $p = .005$; $n = 99$). Intermediate and advanced trends are flat ($p = .33$, $p = .40$), and advanced readers remain the most accurate under obfuscation. Much of the pooled loss is thus contributed by beginners. Confidence is itself experience-graded and badly calibrated—the most confident answers top out near 0.65 accuracy—giving a pool of confident-but-incorrect answers that is the raw material for the System-1 renaming-error signature we develop in RQ2.

RQ1 Takeaway (Partial Replication). We replicate Paper 1’s direction: obfuscation reduces accuracy, most in JavaScript and among beginners. But renaming (L1b) shows no reliable pooled effect (CI crosses 1) — not because it does nothing, but because its effect reverses by item type. Accuracy holds or rises on S1 items (solvable by pattern recognition) and drops on S2 items (which require tracing), so the two cancel. Only control-flow (L2, OR = 0.57) carries a reliable penalty — the route-dependence motivating RQ2.

VI. RQ2: THE TWO CONFUSION ROUTES

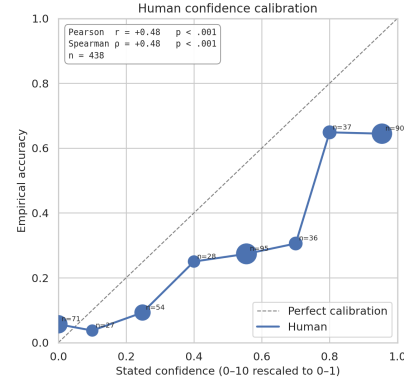
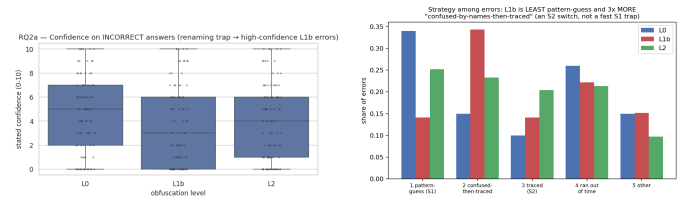


Fig. 8: Calibration curve of empirical accuracy vs. stated confidence (rescaled to 0–1), pooled across conditions. Marker area is proportional to response count.

Readers are miscalibrated in aggregate (Fig. 8): empirical accuracy rises with stated confidence but never reaches the diagonal, and even the most confident answers top out near 0.65 ($r = 0.48$, $p < .001$; $n = 438$). This pool of confident-but-wrong answers could arise two ways—a renaming-specific *trap* that manufactures confident errors (the signature Paper 2 reports in models), or a shared difficulty both obfuscations are *detected* as, pushing readers off their fast heuristic default into slower, effortful processing. The data favor the second. Renaming (L1b) and control-flow (L2) lower accuracy and confidence by similar amounts; what separates them is not confidence versus effort but *which* effortful response each triggers—renaming as Stroop-like interference readers notice and suppress (§VI-A), control-flow as sustained reconstruction of execution state (§VI-B).

A. RQ2a: Renaming as Interference, Not a Confident Trap

If renaming sprang a System 1 trap—as it does in models (Paper 2)—its errors would be *confident* errors: a high-confidence tail of wrong answers that survives because readers never detect the misread. In humans, they do not.



(a) Distribution of stated confidence on incorrect answers by obfuscation level (L0, L1b, L2). (b) Self-reported answering strategy on incorrect answers, by obfuscation level. Renaming (L1b) shows the lowest fast-pattern-guess share and the highest “confused-by-names, then traced” share.

Fig. 9: Confidence and answering strategy on incorrect answers by obfuscation level.

Confidence on incorrect answers is *lower* under renaming, not higher: the L1b median sits below L0 (Fig. 9a), and the high-confidence tail is no heavier than baseline. A participant-clustered logistic model of the high-confidence-incorrect (HCI) rate finds no renaming-specific confident tail (L1b vs. L0 odds ratio 0.85, $p = .67$; the only reliable mover is item route). Restricting to genuine responses—dropping ran-out-of-time non-answers, which concentrate at L1b—mean error confidence is significantly *below* baseline at the renaming tier (-1.05 , $p = .004$) and indistinguishable from control-flow ($p = .93$). Renaming degrades confidence, just like control-flow, rather than manufacturing a confident misread.

The mechanism is the reverse of a fast trap (Fig. 9b). Among errors, renaming *suppresses* fast pattern-guessing rather than inviting it (L1b vs. L0 odds ratio 0.32, $p = .001$) and roughly *triples* the “confused-by-names, then traced” strategy (3.22, $p = .003$)—the behavioural signature of Stroop-like interference, where the adversarial name is recognised, flagged as misleading, and suppressed at the cost of an effortful switch to line-by-line tracing. Renaming errors are thus predominantly errors of *effortful* reading defeated by interference, not fast heuristic misreading. A grader reading only the written trace instead places the largest heuristic share at L1b, matching the model side, because it cannot separate a name never detected from one detected and suppressed; we weight self-reports here and resolve the divergence in RQ4.

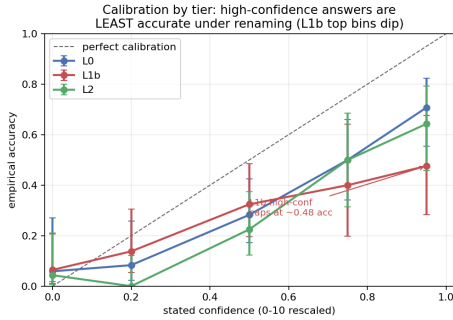
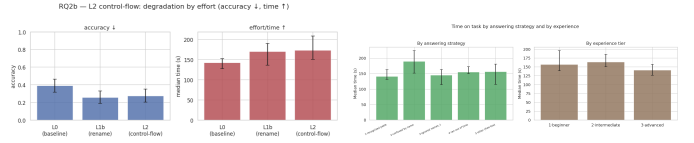


Fig. 10: Per-tier calibration: empirical accuracy vs. stated confidence (Wilson 95% CIs). Overall miscalibration is comparable across tiers, but the top confidence bins are markedly less accurate under renaming.

One narrow trap-like signal survives. Although overall miscalibration is flat across tiers, the *high-confidence* end is not: answers given at top confidence under renaming top out near 0.48 accuracy versus 0.64–0.71 at L0/L2 (Fig. 10)—a *localised* overconfidence rather than a broad confident tail. We flag this cautiously: the top-confidence cells are small ($n \approx 21$) and overlap L0/L2, so it is suggestive rather than established.

B. RQ2b: Control-Flow as System 2 Overload

The second route also forces effort, but a different kind.



(a) Accuracy (left) and median response time (right) by obfuscation level. L1b and L2 both extract more effort for less comprehension. (b) Median time on task by answering strategy (left) and by prior-experience tier (right).

Fig. 11: Control-flow forces effort, and that effort tracks strategy. Error bars are 95% CIs.

Control-flow lowers accuracy while raising time (Fig. 11a; accuracy 0.39 → 0.26 → 0.28 at L0/L1b/L2, median time rising from roughly 145 s at L0 to 170 s at L2). On these axes L1b and L2 are nearly indistinguishable—both extract more effort for less comprehension—so accuracy and time do not separate the two routes. What separates them is the *strategy* the effort is spent on: under renaming it goes into suppressing a misleading name (the confused-then-traced switch of §VI-A), under control-flow into reconstructing execution order through sustained line-by-line tracing.

This cost tracks strategy, not skill (Fig. 11b). Median time varies little across experience tiers but clearly across strategies: the slowest responses are the confused-then-traced ones, with pure tracing and pure pattern-recognition closer together. The temporal cost attaches to deliberative engagement—tracing forced either by interference (L1b) or structural indirection (L2)—which is what licenses reading both obfuscations as effortful rather than fast.

RQ2 Takeaway: Both obfuscation families lower accuracy and confidence by similar amounts through effortful, not fast, processing; the dual route is one of distinct effortful responses. Renaming (L1b) acts as Stroop-like interference—no confident-error tail, but suppressed pattern-guessing and a tripled confused-then-traced switch—while control-flow (L2) spends the same cost on sustained tracing. The clean separation is by strategy, not confidence. This diverges from the model-side HCI signature (Paper 2), though the contrast is partly measure-dependent and left to RQ4.

VII. RQ3: THE LEVERS AND THE COGNITIVE LIMIT

How do time pressure (an S1 lever) and dual-task load (an S2 lever) interact with obfuscation tier to shift strategy, affect accuracy and calibration, and reveal a cognitive limit analogous to Paper 2’s chain-of-thought-length / Goldilocks signal? The two levers turn out to act asymmetrically: time pressure lowers accuracy uniformly, dual-task load only per item, and the per-item view is where the cognitive limit appears.

A. Time Pressure as an S1 Lever (RQ3a)

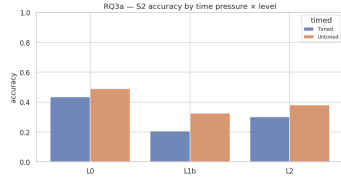


Fig. 12: System-2 accuracy, timed vs. untimed, at L0, L1b, and L2.

Time pressure does the most damage exactly where a fast read is most dangerous. Untimed accuracy exceeds timed accuracy at every tier, but the gap is tier-dependent—smallest at baseline (≈ 0.06 at L0) and widest under renaming (Fig. 12), the regime in which a pattern-driven read is most likely to be wrong. Conditioning the lever on experience is consistent with time pressure costing the least-fluent readers most, but the per-cell CIs overlap and the effect is non-monotone, so we read it as weak corroboration of the experience-graded penalty from RQ1c rather than a clean gradient.

B. Dual-Task Load as an S2 Lever (RQ3b, RQ3d)

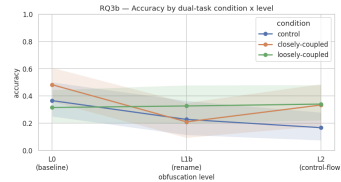


Fig. 13: Accuracy by dual-task (coupling) condition across obfuscation levels. 95% CI bands (shaded).

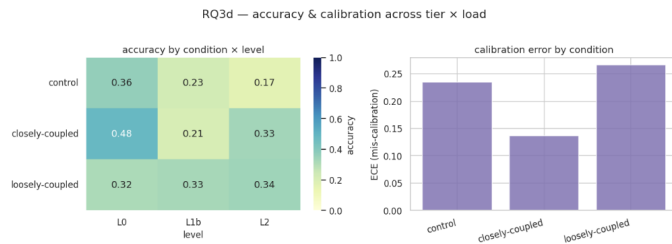


Fig. 14: Accuracy (left) and calibration error / ECE (right) across obfuscation tier and dual-task load.

Dual-task load produces no clean aggregate accuracy effect. The control group declines roughly monotonically ($0.37 \rightarrow 0.23 \rightarrow 0.17$), but the coupled groups do not track obfuscation—closely-coupled dips at L1b and rebounds at L2, loosely-coupled stays flat—and the CI bands overlap throughout (Fig. 13), so these are better read as item-level entanglement than as reliable group differences (we resolve this per item in Fig. 16). Calibration, by contrast, *does* respond to load: closely-coupled load yields the lowest ECE even though accuracy does not move monotonically (Fig. 14). Load

sharpens calibration without uniformly lowering accuracy—the signature of an effect entangled with item difficulty rather than acting cleanly across the board.

C. Strategy Shifts Across Conditions (RQ3c)

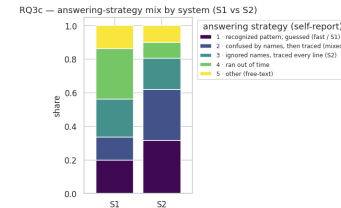


Fig. 15: Self-reported answering-strategy mix for System-1 vs. System-2 questions.

Readers do shift strategy with task demands, but not as a clean System-1 $\rightarrow$ System-2 substitution. From S1 to S2 items the mixed “confused-by-names, then traced” share grows most and “ran out of time” falls, while pure guessing actually *rises* and pure tracing stays flat (Fig. 15): tracing effort increases chiefly through the mixed route, yet many readers still default to a fast pattern-read. This route choice—not reader experience—is what predicts accuracy. The strategy mix is roughly stable across experience tiers, so we do not lean on an expert-traces / novice-guesses gradient; instead, as already shown in RQ1 (Fig. 4), pattern-recognition and tracing trade places by item with neither uniformly best—the item-dependent structure the per-question analysis makes explicit next.

D. Evidence of a Cognitive Limit (RQ3b, RQ3e)

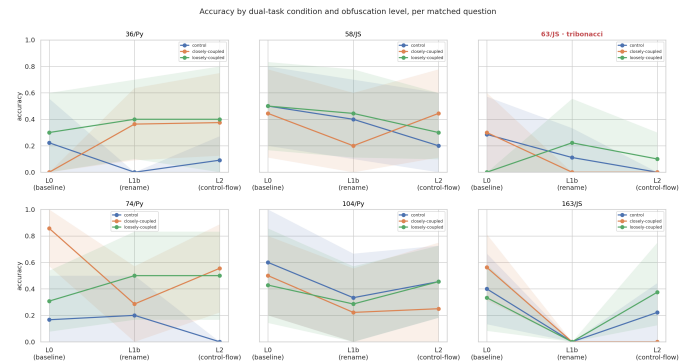


Fig. 16: Accuracy by dual-task condition and obfuscation level for each matched question. The pooled trend in Fig. 13 is driven by a few items; the pattern-recognizable Tribonacci item diverges from tracing-dependent ones. Shaded bands are 95% CIs.

Disaggregating the dual-task effect by item resolves the messy aggregate (Fig. 16): the pooled trend of Fig. 13 is driven by a few items, not a uniform load response. The pattern-recognizable Tribonacci item responds to load oppositely from tracing-dependent items—the item-level Goldilocks /

cognitive-limit structure, and the human analog of Paper 2’s chain-of-thought-length effect.

RQ3 Takeaway: *The two levers act asymmetrically. Time pressure (an S1 lever) lowers accuracy at every tier, most under renaming. Dual-task load (an S2 lever) shows no clean aggregate accuracy effect—only sharper calibration—and makes sense only per item. Readers add tracing on tracing-demanding items, but via a mixed confused-then-traced route rather than a clean S1→S2 switch. The per-item view exposes a Goldilocks limit: pattern-recognizable items (Tribonacci) respond to load oppositely from tracing-dependent ones.*

VIII. RQ4: MODEL ALIGNMENT

Do large language models reproduce the human dual-route response to obfuscation? Scoring both populations on a common output-prediction task and matched stimuli, we find a single pattern across every axis: models reproduce the *process* of human difficulty—its accuracy shape (RQ4a), the deliberate effort it recruits (RQ4c), and its bounded tracing budget (RQ4d)—but not its *magnitude*, as accuracy level, control-flow’s accuracy cost, and confidence resolution all fail to transfer. The one apparent *qualitative* divergence—renaming, where humans and models look opposite—is the exception that proves the rule: it dissolves once both populations are scored on a shared instrument (RQ4b), the result that closes the loop with Section VI.

A. RQ4a: Shape Transfers, Level Does Not

Accuracy (Fig. 17). The model trace follows the human shape rather than a textbook monotonic decline: every family dips at L1b and recovers at L2 (pooled $0.42 \rightarrow 0.29 \rightarrow 0.37$), with dip depth family-dependent. The by-language panel carries the human asymmetry—JavaScript degrades more sharply than the comparatively flat Python—and the experience panel places the model mean above even the advanced human tier at all three tiers by a near-constant gap. The shape of the obfuscation response transfers while the level does not.

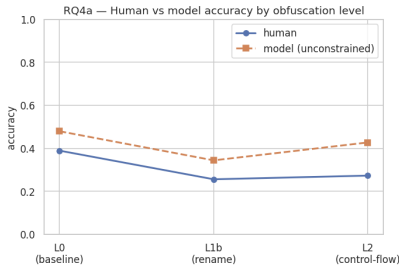


Fig. 17: Human and model accuracy by obfuscation level (L0, L1b, L2), split by model family (left), language (center), and human experience tier vs. the model mean (right); 95% CIs.

Confidence does not transfer (Fig. 18). Models report no confidence, so we read out four probability proxies. All sit

below the diagonal, but their failure modes differ from the human one and from each other: the human curve resolves at the top (climbing to ≈ 0.65), whereas the proxies stay flat, conservative, and mutually disagreeing on shape (e.g. $\exp(\log\text{prob})$ tracks the human curve while $1/\text{perplexity}$ and $1 - \text{entropy}$ hug zero until very high confidence). Confidence *resolution* therefore does not transfer, and the conclusion depends on which internal signal is called “confidence.” This is why the renaming signature cannot be a *confidence* signature on the model side—it must be read from reasoning mode, which is the premise RQ4b builds on.

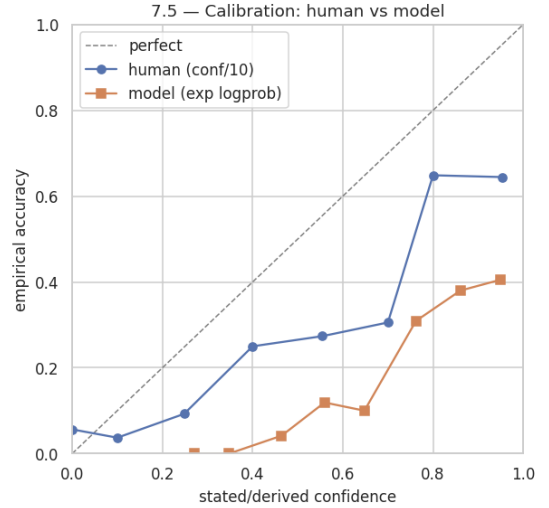


Fig. 18: Calibration curves of empirical accuracy vs. confidence for the human (conf/10) and four model proxies— $\exp(\log\text{prob})$, self-certainty, $1/\text{perplexity}$, $1 - \text{entropy}$ —against the perfect-calibration diagonal.

B. RQ4b: The Renaming Divergence Is a Measurement Artifact

Section VI left an apparent contradiction. Under adversarial renaming, *models* fall for the trap—their errors are heuristic, System-1 errors—whereas *humans* appear to escape it, reporting that the misleading name made them *stop* guessing and start tracing (the “confused-then-traced” switch). We show this opposition is not a behavioral difference but an artifact of measuring the two populations with two different instruments.

The two instruments ask different questions. *Self-report* records the strategy a reader consciously adopted, so detecting a bad name and switching to tracing is logged as a deliberate System-2 move—hiding the heuristic pull that triggered it. The *grader* sees only the written trace and, per our codebook, infers mode from that text; it cannot distinguish a name that was never detected from one that was detected and suppressed, because both leave a fast, pattern-shaped trace on the page. Humans were scored mostly by the first ruler and models entirely by the second. Put both populations on the *grader* ruler and the divergence disappears (Fig. 19): human heuristic share, which *dips* at L1b under self-report ($0.40 \rightarrow 0.24$), instead *peaks* at L1b under grader coding ($0.38 \rightarrow 0.51$),

matching the model’s own L1b peak ($0.71 \rightarrow 0.77$). The same human responses yield opposite pictures at the renaming tier depending only on the ruler.

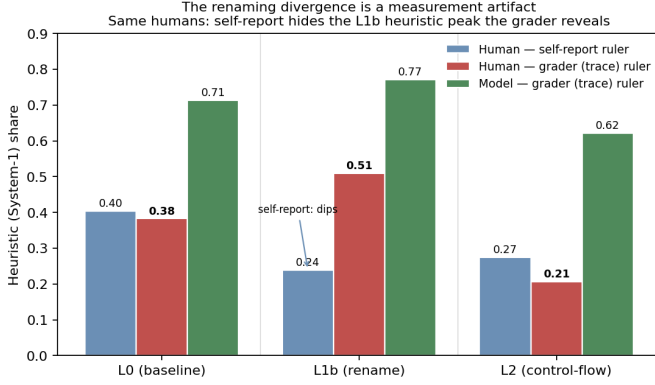


Fig. 19: Heuristic (System-1) share by obfuscation tier under two scoring rulers: human self-report, human grader (mode inferred from the written trace), and model grader.

Errors confirm the mechanism. Conditioning the model mode mix on correctness, incorrect responses carry a larger heuristic share than correct ones at every tier, and the gap is widest at L1b (0.84 vs. 0.65): wrong answers under renaming are System-1 errors. The trap is not merely a compute shortage—within the System-2 budget sweep, L1b error persists at the largest budgets that drive L0/L2 errors down (Appendix Fig. 20)—the model-side echo of human renaming errors that effort does not fully resolve. We report this renaming alignment as a *mode-based* analog of the trap, scored on a shared instrument, not as a reproduction of a human high-confidence signature (which, per RQ4a, does not exist on the model side).

Fig. 20: Total error counts by tier and token condition; L1b error persists at the largest budgets.

Statistical note. The self-report L1b dip is reliable (L1b vs. L0 heuristic-share, Fisher $p = .005$, $n = 389$); the grader L1b peak is in the predicted direction but does not reach significance on the L1b-vs-L0 contrast alone ($p = .14$, $n = 233$, grader coding available for the VALID subset). The qualitative reversal between rulers—a dip under one and a peak under the other on the same responses—is the load-bearing result; we treat the grader peak’s magnitude as suggestive and a target for the larger graded sample.

C. RQ4c: Control-Flow Aligns on Effort, Not Cost

The prompt lever is genuine, not a relabeling (Fig. 21): instructed System 1 yields almost entirely heuristic stacks at every tier, while instructed System 2 grows the tracing share monotonically from L0 to L2. So models, like humans, recruit more deliberate tracing as obfuscation increases, with

the shift concentrated at L2—the *effort* half of the control-flow signature. But this addresses only half of RQ4c: model accuracy *recovers* at L2 rather than degrading (Fig. 17), so the System-2-overload *degradation* predicted for control-flow does not appear on the model side. Control-flow aligns on the effort it recruits, not on the accuracy cost it imposes in humans—the same process-without-magnitude split seen for the accuracy shape in RQ4a.

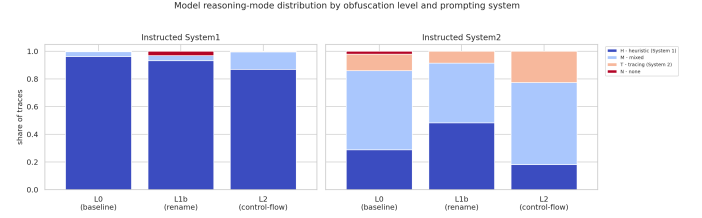


Fig. 21: Model reasoning-mode distribution by obfuscation tier under instructed System-1 vs. instructed System-2 prompting.

D. RQ4d: The Cognitive Limit Is a Bounded Budget

The System-2 accuracy curve rises from a low base at 256 tokens (0.08), climbs through 512–1024, and flattens around 2048 (0.42), after which 10,000 tokens add essentially nothing (0.45); the System-1 curve is flat throughout (Fig. 22), so the effect is specific to deliberate tracing. This is a saturation / diminishing-returns plateau, not an inverted-U—within the tested range accuracy does not fall back down. Once more the *process* transfers but its precise form does not: a bounded effective reasoning budget, a loose analog of the human Goldilocks band rather than a matching curve shape.

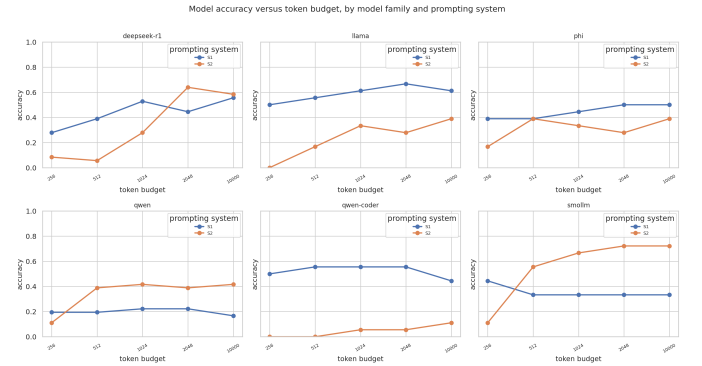


Fig. 22: Model accuracy vs. token budget (256–10,000) for System-1 vs. System-2 prompting.

E. Scale and Tuning

Across the family, size is the dominant axis: sub-billion models sit near floor at all tiers while the 7B reasoning-distilled model reaches the top of the range and stays high under obfuscation (Fig. 23). The size–accuracy trend is strongly monotone but underpowered (model-level Spearman $\rho = 0.54$, $p = .17$, $n = 8$ models; the response-level $\rho = 0.91$ over-counts by treating one model’s responses as independent). Two off-trend deviations carry the tuning point: the reasoning-distilled 7B

model punches above its parameter count, and the coder-tuned 7B under-performs same-size peers and collapses toward floor at L1b. We frame tuning as a deviation from the size trend rather than an independent main effect, and note it leans on a single reasoning-distilled checkpoint—a statement about available models, not a general property of reasoning tuning.

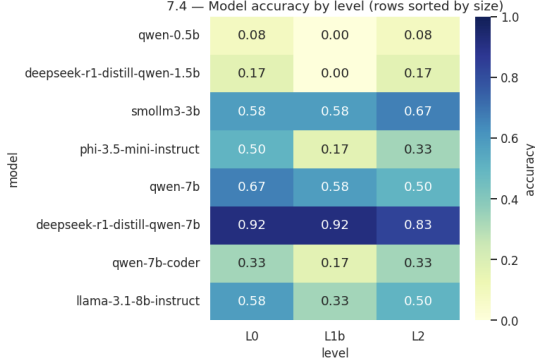


Fig. 23: Model accuracy by obfuscation tier grouped by family and parameter count (left) and accuracy vs. parameter count on a log scale (right).

RQ4 Takeaway: *Models reproduce the process of human difficulty but not its magnitude. The accuracy shape (L1b dip, L2 recovery), the deliberate effort control-flow recruits, and a bounded tracing budget all transfer; accuracy level, control-flow’s accuracy cost, and confidence resolution do not. The one apparent qualitative divergence is an artifact: humans were scored by self-report (crediting the deliberate confused-then-traced switch), models by a grader reading the trace (which cannot see it); on a shared grader ruler both peak in heuristic reasoning at L1b. Within this picture, accuracy rises with size and favors reasoning-distilled over coder/instruct tuning.*

IX. THREATS TO VALIDITY

Sample and Measurement. Our 73 participants are a modest, self-selected sample with large between-person heterogeneity that can swamp condition effects; several contrasts (renaming, language, coupling) have CIs crossing 1 and may be underpowered rather than null. Experience uses a coarse three-level self-report, and our confidence and strategy measures are post-hoc self-reports subject to recall and rationalization bias, making the dual-process labels interpretive rather than observed. *Stimuli and Task Design.* Items are deterministic, self-contained HumanEval-X functions (<15 lines, cyclomatic complexity 4–8) in Python and JavaScript only, omitting real-world complexity (external dependencies, larger scopes, unfamiliar domains). Output prediction imperfectly proxies comprehension, and each tier instantiates one renaming and one control-flow tool, so results may not generalize to other transformation families. Item-level claims rest on few matched snippets and are illustrative. *Model Evaluation.* HumanEval-X

is widely published, so training-data contamination may inflate model accuracy and partly explain the human–model offset. Lacking reported confidence, our calibration comparison relies on disagreeing probability-based proxies, so proxy choice materially affects alignment conclusions. Finally, the model panel spans limited architectures and sizes, so the “reasoning-tuned aligns, coder/instruct does not” result reflects available checkpoints, not an exhaustive sweep.

X. RELATED WORK

Obfuscation and Human Code Comprehension. [4] provides a taxonomy motivating our tiered design, and surveys establish obfuscation as a controlled probe of model robustness [8], [9]. Identifier renaming reduces task performance [10]; comprehension draws on working memory and semantic cues [11], [12], with identifier quality modulating cognitive load [13], [14]. Most directly, Nguyen *et al.* [1] measured human comprehension under these obfuscation tiers; we build on their paradigm and data to study whether LLMs exhibit the same difficulty patterns.

LLM Reasoning and Cognitive Alignment. CoT prompting improves performance on difficult tasks [15], [16], [17], and reasoning effort scales with task difficulty [18], [19]. Reasoning traces may be post-hoc rather than faithful [20], [21], and token-level uncertainty is linked to confusion in humans and models [22]. A separate line relates human visual attention from eye movements to code models—improving neural code summarization [23], [24], mimicking developer attention [25], and testing model–developer alignment [26]; we draw on it only as motivation for scoring human and model comprehension on a shared instrument.

Expert–Novice Differences in Program Comprehension. Experts reason top-down via plan-based, hierarchical representations that integrate control flow and goals, whereas novices process code locally, line by line [27], [28], [29], anchoring on identifiers as semantic “beacons” [30].

Identifier Sensitivity and Adversarial Robustness in Code Models. Identifiers carry rich semantics in programs [31], [32], yet adversarial renaming misleads models [33], [34], [35]. Code models often rely on surface cues over deep semantics [36], and identifier quality affects performance [37].“

XI. CONCLUSION

This paper shows that code obfuscation disrupts comprehension through different cognitive routes rather than a single monotonic difficulty effect. Adversarial renaming creates misleading surface cues that trigger interference, while control-flow obfuscation increases the cost of sustained tracing. Our human study shows that the strongest reliable penalty comes from control-flow obfuscation, especially for less-experienced readers and JavaScript tasks. The human–model comparison further shows that reasoning-tuned LLMs reproduce the overall shape of human difficulty, but not its full magnitude or confidence behavior. These findings highlight obfuscation as a useful probe for diagnosing fragile, surface-driven code understanding in both humans and models.

Data Availability Statement: Our package is available at [38].

REFERENCES

- [1] A. H. N. Nguyen, J. Le, I. L. Coronado, and T. N. Nguyen, "The effect of code obfuscation on human program comprehension," 2026. [Online]. Available: <https://arxiv.org/abs/2603.07668>
- [2] C. Schulte, "Block model: an educational model of program comprehension as a tool for a scholarly approach to teaching," in *Proceedings of the Fourth International Workshop on Computing Education Research*, ser. ICER '08. New York, NY, USA: Association for Computing Machinery, 2008, p. 149–160. [Online]. Available: <https://doi.org/10.1145/1404520.1404535>
- [3] J. Le, A. H. N. Nguyen, and T. N. Nguyen, "Do machines struggle where humans do? LLM and human comprehension of obfuscated code." [Online]. Available: <https://arxiv.org/abs/2606.31725>
- [4] C. Collberg, C. Thomborson, and D. Low, "A taxonomy of obfuscating transformations," University of Auckland, Tech. Rep. 148, 07 1997. [Online]. Available: <https://researchspace.auckland.ac.nz/handle/2292/3491>
- [5] M. Ceccato, M. Di Penta, J. Nagra, P. Falcarin, F. Ricca, M. Torchiano, and P. Tonella, "Towards experimental evaluation of code obfuscation techniques," in *Proceedings of the 4th ACM Workshop on Quality of Protection*, ser. QoP '08. New York, NY, USA: Association for Computing Machinery, 2008, pp. 39–46. [Online]. Available: <https://doi.org/10.1145/1456362.1456371>
- [6] T. Laszlo and A. Kiss, "Obfuscating c++ programs via control flow flattening," vol. 30, 06 2007.
- [7] J. S. B. T. Evans and K. E. Stanovich, "Dual-process theories of higher cognition: Advancing the debate," *Perspectives on Psychological Science*, vol. 8, no. 3, pp. 223–241, 2013, pMID: 26172965. [Online]. Available: <https://doi.org/10.1177/1745691612460685>
- [8] S. Schrittwieser, S. Katzenbeisser, J. Kinder, G. Merzdovnik, and E. Weippl, "Protecting software through obfuscation: Can it keep pace with progress in code analysis?" *ACM Comput. Surv.*, vol. 49, no. 1, Apr. 2016. [Online]. Available: <https://doi.org/10.1145/2886012>
- [9] S. Banescu, C. Collberg, and A. Pretschner, "Predicting the resilience of obfuscated code against symbolic execution attacks via machine learning," in *26th USENIX Security Symposium (USENIX Security 17)*. Vancouver, BC: USENIX Association, Aug. 2017, pp. 661–678. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/banescu>
- [10] M. Ceccato, M. Penta, P. Falcarin, F. Ricca, M. Torchiano, and P. Tonella, "A family of experiments to assess the effectiveness and efficiency of source code obfuscation techniques," *Empirical Softw. Engg.*, vol. 19, no. 4, p. 1040–1074, Aug. 2014. [Online]. Available: <https://doi.org/10.1007/s10664-013-9248-x>
- [11] J. Siegmund, C. Kästner, S. Apel, C. Parnin, A. Bethmann, T. Leich, G. Saake, and A. Brechmann, "Understanding understanding source code with functional magnetic resonance imaging," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 378–389. [Online]. Available: <https://doi.org/10.1145/2568225.2568252>
- [12] J. Siegmund, N. Peitek, C. Parnin, S. Apel, J. Hofmeister, C. Kästner, A. Begel, A. Bethmann, and A. Brechmann, "Measuring neural efficiency of program comprehension," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017. New York, NY, USA: Association for Computing Machinery, 2017, p. 140–150. [Online]. Available: <https://doi.org/10.1145/3106237.3106268>
- [13] D. Lawrie, C. Morrell, H. Feild, and D. Binkley, "What's in a name? a study of identifiers," in *14th IEEE International Conference on Program Comprehension (ICPC'06)*, 2006, pp. 3–12.
- [14] D. Binkley, M. Davis, D. Lawrie, J. Maletic, C. Morrell, and B. Sharif, "The impact of identifier style on effort and comprehension," *Empirical Software Engineering*, vol. 18, 04 2013.
- [15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [16] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2203.11171>
- [17] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [18] A. G. de Varda, F. P. D'Elia, H. Kean, A. Lampinen, and E. Fedorenko, "The cost of thinking is similar between large reasoning models and humans," *Proceedings of the National Academy of Sciences*, vol. 122, no. 47, p. e2520077122, 2025.
- [19] D. Yan, Z. Gao, and Z. Liu, "A closer look at different difficulty levels code generation abilities of chatgpt," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23. IEEE Press, 2024, p. 1887–1898. [Online]. Available: <https://doi.org/10.1109/ASE56229.2023.00096>
- [20] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," 2023. [Online]. Available: <https://arxiv.org/abs/2305.04388>
- [21] T. Lanham, A. Chen, A. Radhakrishnan, B. Steiner, C. Denison, D. Hernandez, D. Li, E. Durmus, E. Hubinger, J. Kernion, K. Lukošiušė, K. Nguyen, N. Cheng, N. Joseph, N. Schiefer, O. Rausch, R. Larson, S. McCandlish, S. Kundu, S. Kadavath, S. Yang, T. Henighan, T. Maxwell, T. Telleen-Lawton, T. Hume, Z. Hatfield-Dodds, J. Kaplan, J. Brauner, S. R. Bowman, and E. Perez, "Measuring faithfulness in chain-of-thought reasoning," 2023. [Online]. Available: <https://arxiv.org/abs/2307.13702>
- [22] Y. Abdelsalam, N. Peitek, A.-M. Maurer, M. Toneva, and S. Apel, "How do humans and llms process confusing code?" 2025. [Online]. Available: <https://arxiv.org/abs/2508.18547>
- [23] A. Bansal, B. Sharif, and C. McMillan, "Towards modeling human attention from eye movements for neural source code summarization," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, no. ETRA, pp. 1–19, May 2023. [Online]. Available: <http://dx.doi.org/10.1145/3591136>
- [24] Y. Zhang, J. Li, Z. Karas, A. Bansal, T. J.-J. Li, C. McMillan, K. Leach, and Y. Huang, "Eyetrans: Merging human and machine attention for neural code summarization," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: <https://doi.org/10.1145/3643732>
- [25] Y. Zhang, C. Huang, Z. Karas, T. D. Nguyen, K. Leach, and Y. Huang, "Enhancing code llm training with programmer attention," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '25. ACM, Jun. 2025, pp. 616–620. [Online]. Available: <http://dx.doi.org/10.1145/3696630.3728510>
- [26] M. Paltenghi and M. Pradel, "Thinking like a developer? comparing the attention of humans with neural models of code," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Nov. 2021, pp. 867–879. [Online]. Available: <http://dx.doi.org/10.1109/ASE51524.2021.9678712>
- [27] E. Soloway and K. Ehrlich, "Empirical studies of programming knowledge," *IEEE Trans. Softw. Eng.*, vol. 10, no. 5, p. 595–609, Sep. 1984. [Online]. Available: <https://doi.org/10.1109/TSE.1984.5010283>
- [28] V. Fix, S. Wiedenbeck, and J. Scholtz, "Mental representations of programs by novices and experts," in *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems*, 1993, pp. 74–79.
- [29] N. Pennington, "Stimulus structures and mental representations in expert comprehension of computer programs," *Cognitive Psychology*, vol. 19, no. 3, pp. 295–341, 1987. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0010028587900077>
- [30] C. L. CORRITORE and S. WIEDENBECK, "An exploratory study of program comprehension strategies of procedural and object-oriented programmers," *International Journal of Human-Computer Studies*, vol. 54, no. 1, pp. 1–23, 2001. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1071581900904233>
- [31] M. Allamanis, E. T. Barr, C. Bird, and C. Sutton, "Suggesting accurate method and class names," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 38–49. [Online]. Available: <https://doi.org/10.1145/2786805.2786849>
- [32] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," 2018. [Online]. Available: <https://arxiv.org/abs/1711.00740>

- [33] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, and Z. Jin, "Generating adversarial examples for holding robustness of source code processing models," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, pp. 1169–1176, Apr. 2020. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/5469>
- [34] N. Yefet, U. Alon, and E. Yahav, "Adversarial examples for models of code," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020. [Online]. Available: <https://doi.org/10.1145/3428230>
- [35] J. Henkel, G. Ramakrishnan, Z. Wang, A. Albarghouthi, S. Jha, and T. Reps, "Semantic robustness of models of source code," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2022, pp. 526–537.
- [36] M. R. I. Rabin, N. D. Bui, K. Wang, Y. Yu, L. Jiang, and M. A. Alipour, "On the generalizability of neural program models with respect to semantic-preserving program transformations," *Information and Software Technology*, vol. 135, p. 106552, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584921000379>
- [37] S. L. Nikiema, J. Samhi, A. K. Kaboré, J. Klein, and T. F. Bissyandé, "The code barrier: What llms actually understand?" 2025. [Online]. Available: <https://arxiv.org/abs/2504.10557>
- [38] Anonymous, "Fast errors or slow effort? divergent dual-process signatures of code obfuscation in humans and language models," 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.21076966>